NanoObjectDetection 2.0

1. Purpose

NanoObjectDetection 2.0 is the compatibility implementation of the original NanoObjectDetection 1.0 analysis. It preserves the 1.0 parameter names, physical conventions, analysis stages, trajectory logic, and legacy output names while adding two execution backends:

? bounded-memory chunked TIFF processing;

? optional CUDA preprocessing and uint16 conversion.

The 1.0 package remains available in `nanoobjectdetection_1_0/`. The independent pipeline that was formerly called the modern pipeline is version 3.0 and is documented separately.

The design principle is **same scientific settings, different execution backend**. Changing from `single_stack` to `chunked` should not require changing `parameter.json`. GPU processing is an acceleration of preprocessing, not a different tracking algorithm.

2. Quick start

Install CPU dependencies:

```
/path/to/python -m pip install -r /path/to/NanoObjectDetection/requirements-2.0.txt
```

Run the example in chunked CPU mode:

```
/path/to/python /path/to/NanoObjectDetection/main.py \
  --parameter-file /path/to/example/parameter.json
```

Run the full-stack compatibility backend:

```
/path/to/python /path/to/NanoObjectDetection/main.py \
  --parameter-file /path/to/example/parameter.json \
  --processing-mode single_stack
```

Run with GPU preprocessing:

```
/path/to/python /path/to/NanoObjectDetection/main.py \
  --parameter-file /path/to/example/parameter.json \
  --processing-mode chunked --gpu
```

GPU mode requires a compatible CuPy package and CUDA runtime. The program fails explicitly if `--gpu` is requested but no CUDA device is available.

3. Processing stages

The 2.0 workflow follows the same conceptual order as 1.0:

? Read the TIFF stack and apply the configured frame and spatial ROI.

? Convert working data to float32.

? Optionally remove camera offset.

? Optionally normalize laser fluctuations frame by frame.

? Estimate and subtract the temporal static background.

? Optionally apply Gaussian SNR enhancement.

? Scale to uint16 for Trackpy.

? Localize particles with Trackpy.

? Link detections into moving and stationary trajectories.

? Remove detections in stationary-particle no-go regions.

? Split or reject trajectories according to `Split`.

? Estimate global drift and subtract accumulated drift.

? Calculate MSD and fit diffusion.

? Convert diffusion to hydrodynamic diameter.

? Apply optional hindrance and diameter filters.

? Write 2.0 tables and 1.0-compatible aliases.

? Create legacy-style plots and export plot data.

Chunked mode stores source frames in `image_store/` and processes one block at a time. Before localization, chunked mode makes a bounded-memory background pass over all chunks and writes one `global_background.npy` image. That same background is subtracted from every chunk, and is also used during global intensity scaling. The global image is the median of the per-chunk temporal background estimates; this avoids loading the complete TIFF stack while preventing independent background levels at chunk boundaries. Linking and drift correction still operate on the combined detection and trajectory tables, so drift is not estimated independently per chunk.

4. Version 1.0 parameter compatibility

4.1 `Exp`

Parameter	Meaning
`NA`	Numerical aperture. Stored as experiment metadata.
`lambda`	Illumination wavelength in nm. Stored as metadata.
`fps`	Acquisition frame rate. Used for time axes and MSD.
`ExposureTime`	Exposure time in seconds. Stored as metadata; no motion-blur correction is currently applied.
`Microns_per_pixel`	Spatial calibration used for diffusion and diameter.
`Temperature`	Temperature in K.
`Viscosity`	1.0 viscosity convention, converted to Pa s internally.

Wavelength, NA, and exposure time are deliberately preserved but do not yet alter localization precision or the MSD fit. They are required for complete experimental provenance and future optical corrections.

4.2 `Fiber`

Parameter	Meaning
`TubeDiameter_nm`	Nominal channel diameter for hindrance correction.
`Center_y_px`	Channel centre in pixels, or `"unknown"` for automatic estimation.
`Mode`, `Waist`	Preserved experiment descriptors. The current position model assumes a circular channel and does not use Gaussian intensity fitting.

4.3 `ROI`

`x_min`, `x_max`, `y_min`, `y_max` define the spatial slice. `frame_min` and `frame_max` define the half-open frame interval. Large sentinel maxima are interpreted as “until the end”.

4.4 `PreProcessing`

Parameter	Effect
`Remove_CameraOffset`	Subtract the minimum intensity of the working data.
`Remove_Laserfluctuation`	Normalize each frame by its mean intensity relative to the median frame intensity.
`Remove_StaticBackground`	Estimate one global temporal background across all chunks and subtract it consistently.
`BackgroundCores`	Retained for 1.0 compatibility. Chunked 2.0 controls memory through chunk size instead.
`AllowNonPosValues`	Trackpy input is clipped/scaled to positive uint16 values.
`EnhanceSNR`	Apply the 1.0-compatible Gaussian spatial smoothing.

The default static background is the trimmed temporal mean: values between approximately the 30th and 70th percentile are averaged independently for each pixel. In chunked mode this is computed once per chunk and the

resulting background images are combined by their pixelwise median. This suppresses transient particles while retaining stationary background structure, without requiring the complete video in memory. The saved `global_background.npy` file is an intermediate diagnostic and can be deleted before a new run.

4.5 `Find`

Parameter	Effect
`tp_diameter`	Odd Trackpy feature diameter in pixels.
`tp_minmass`	Integrated feature-mass threshold.
`tp_separation`	Minimum separation between features.
`tp_percentile`	Percentile threshold used by Trackpy.

These values retain the 1.0 intensity scaling convention. A threshold tuned for another preprocessing method is not interchangeable.

4.6 `Link` and stationary objects

`Dark time` is passed to Trackpy as memory. `Max displacement` is the primary search range. `Max displacement fix` and `Dwell time stationary objects` control the stationary-particle pass. `Min tracking frames before drift` is applied before drift estimation; `Min_tracking_frames` is applied afterward.

When fixed spots are enabled, detections close to their trajectories are removed before moving-particle linking. This preserves the 1.0 no-go-area behavior but can remove genuine slow particles near a fixed bright object.

4.7 `Split`

`IntensityJump` enables intensity discontinuity splitting. `Max rel median intensity step` controls the relative jump threshold. `Max_traj_length` limits trajectory length; a very large 1.0 sentinel means unlimited. `Max_traj_length_keep_tail` controls whether a short final fragment is retained.

4.8 `Drift`

`Apply` enables or disables drift correction. In global mode, 2.0 uses the same Trackpy `compute_drift` operation as 1.0, with the configured `Avg frames`/rolling smoothing semantics. Missing frame values are linearly interpolated before subtraction.

The exported drift table contains:

```
frame, drift_x, drift_y
```

The legacy plot displays `drift_y` as transversal and `drift_x` as along fiber, using accumulated drift in pixels.

`LaminarFlow` remains a compatibility setting. The global compatibility entry point currently uses the global drift path; a dedicated transversal-flow mode requires the 1.0 `Drift.TransversalEstimation` algorithm and should not be silently substituted by a global correction.

4.9 `MSD`

`lagtimes_min` and `lagtimes_max` define the fitted lag range. `CheckBrownianMotion` enables the quality filter. `EstimateHindranceFactor` and `HindrancePositionAware` select the hindrance mode described below.

For one dimension:

```
MSD(tau) = intercept + 2 * D * tau
```

For two dimensions:

```
MSD(tau) = intercept + 4 * D * tau
```

The diffusion coefficient is converted using the Stokes-Einstein relation:

```
d = 2 * kB * T / (6 * pi * eta * D)
```

The fit uses physical time from `fps` and physical distance from `Microns_per_pixel`.

4.10 `PostProcessing` and `Plot`

`MaxDiameter` removes diameters above the configured limit from statistics and plots while retaining their filter reason in the particle table. `Histogramm_abs_or_rel` selects unweighted, weighted, or both histograms. `Histogramm_Bin_size_nm`, `Histogramm_min`, and `Histogramm_max` control legacy histogram binning and axes. `DiamOverTraj` enables the scatter plot. `DiamOverTraj_UseRawMass` selects mean, median, or maximum brightness.

5. Hindrance correction

Three modes are available internally:

`none`

The unconfined Stokes-Einstein diameter is reported.

`legacy`

One fixed `TubeDiameter_nm` is used for every trajectory, independent of transverse position. This is the 1.0 global correction.

`position`

The corrected transverse trajectory position is compared with `Center_y_px`. For a circular channel, the available local diameter is approximated as twice the wall distance. The mean local diameter over each trajectory is then used for the hindrance inversion.

This is a geometric approximation. It is not a measurement of the optical intensity profile and does not use `Fiber.Mode` or `Fiber.Waist`. A trajectory outside the nominal channel is clipped to zero available wall distance; such data should be inspected because the correction becomes physically unreliable.

6. Exact 1.0-compatible output set

The primary 2.0 files retain explicit modern column names. Compatibility aliases are also written:

```
obj_all.csv
traj_fixed.csv
obj_moving_no_go_area.csv
traj_moving.csv
traj_no_drift.csv
traj_final.csv
sizes_df_lin_long.csv
Diameter_Histogramm-long-weighted.csv
Diameter_Histogramm-long-unweighted.csv
DiameterOverTrajLength_long.csv
```

The legacy plot names are:

```
Global_Drift.png
Diameter_Histogramm-long-weighted.png
Diameter_Histogramm-long-unweighted.png
DiameterOverTrajLength_long.png
MSD_fit_long.png
Trajectories_long.png
```

Equivalent `trans` and `2d` files are created for the other evaluated directions. Histograms use the 1.0 bin size, absolute-occurrence y-axis, 10--300 nm default range from the example, and trajectory-length weighting by repeating each diameter once per valid frame.

7. Chunked and GPU limitations

Chunk overlap prevents most boundary-localization losses, but no chunked implementation can guarantee bit-identical Trackpy results to a single full-stack run when numerical scaling or floating-point order changes. Compare detections, trajectory lengths, drift, and diameter distributions when changing `chunk_frames`.

GPU acceleration currently covers preprocessing and uint16 conversion. Trackpy linking and MSD fitting remain CPU operations. GPU output can differ slightly at threshold boundaries because of floating-point operation order.

8. Reproducibility checklist

Record all of the following with each result:

? the exact `parameter.json`;

? processing mode;

? chunk size and overlap;

? GPU flag and CuPy/CUDA version;

? input TIFF filename and frame range;

? output `analysis_config.json`;

? detection and trajectory counts;

? the drift table and the diameter summary.

For a 1.0 comparison, first run 2.0 in `single_stack` mode, then compare the same configuration in `chunked` CPU mode, and only then enable GPU mode.